

Total points = 62

```
In [1]: pts1 = 10
pts2 = 6 + 4 + 8 + 6 + 8
pts3 = 4 + 4 + 2 + 10
print(pts1, pts2, pts3, pts1 + pts2 + pts3)

10 32 20 62
```

Problem 1: install Python, numpy, matplotlib, astropy (10 pts)

full credit if the rest of the assignment is attempted

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import astropy.units as u
import astropy.constants as c
```

Problem 2a: For Beta Pictoris and Gaia DR3 4373465352415301632, gather the mean values for the parallax, apparent brightness in Gaia's G filter, and the mean effective temperature and report them as your answer (6pts)

one point for each value

-0.5 for each missing unit

```
In [3]: beta_pic_parallax = 50.9307 * u.mas
beta_pic_G = 3.8232
beta_pic_teff = 7938 * u.K
```

```
In [4]: GDR3_star_parallax = 2.07 * u.mas
GDR3_star_G = 13.772
GDR3_star_teff = 5396 * u.K
```

Problem 2b: calculate the apparent bolometric magnitude for each star (4 pts)

2pts per bolometric magnitude

```
In [5]: beta_pic_BC = 0.0172
GDR3_star_BC = 0.0035
```

$$M_{\rm{abs}} = M_{\rm{app}} + BC$$

```
In [6]: beta_pic_bol_app = beta_pic_G + beta_pic_BC
        GDR3_star_bol_app = GDR3_star_G + GDR3_star_BC
```

```
In [7]: print(beta_pic_bol_app)
        print(GDR3_star_bol_app)
```

```
3.8404
13.775500000000001
```

The spectral class of the Gaia DR3 star is G type, which is square in the Gaia G filter, while Beta Pictoris is hotter (an A spectral type), so the bolometric correction is larger because less flux is emitted in the G filter temperature range.

Problem 2c: First estimate the absolute bolometric magnitude of each star, using the estimated distance. Then, using the absolute bolometric magnitude of the Sun, calculate the bolometric luminosity of both Beta Pictoris and Gaia DR3 4373465352415301632. (8 pts)

2pts per absolute bolometric magnitude, 2pts per luminosity

-0.5 pts for each missing unit

(distance/parsec) = (arcsec/parallax), but we can also use astropy equivalencies!

```
In [8]: beta_pic_dist = beta_pic_parallax.to(u.pc, equivalencies=u.parallax())
        GDR3_star_dist = GDR3_star_parallax.to(u.pc, equivalencies=u.parallax())

        print(beta_pic_dist)
        print(GDR3_star_dist)
```

```
19.634522989081237 pc
483.0917874396135 pc
```

$$M_{\rm{app}} - M_{\rm{abs}} = 5 \log_{10}(d/10 \text{ pc})$$

```
In [9]: beta_pic_m_abs_bol = beta_pic_bol_app - 5 * np.log10(beta_pic_dist/(10*u.pc))
        GDR3_star_m_abs_bol = GDR3_star_bol_app - 5 * np.log10(GDR3_star_dist/(10*u.pc))

        print(beta_pic_m_abs_bol)
        print(GDR3_star_m_abs_bol)
```

```
2.3752982261634523
5.3553517272845905
```

$$M_{\rm{abs, \odot, bol}} = 4.75$$

$$M_{\rm{abs, \star, bol}} = M_{\rm{abs, \odot, bol}} - 2.5 \log_{10} \left(\frac{L_{\star}}{L_{\odot}} \right)$$

convert to $L_{\star}$

$$M_{\rm{abs},\star} - M_{\rm{abs},\odot} = -2.5 \log_{10} \left(\frac{L_{\star}}{L_{\rm{odot}}} \right)$$

$$10^{-\frac{(M_{\rm{abs},\star} - M_{\rm{abs},\odot})}{2.5}} = \frac{L_{\star}}{L_{\rm{odot}}}$$

$$L_{\star} = L_{\rm{odot}} \times 10^{-\frac{(M_{\rm{abs},\star} - M_{\rm{abs},\odot})}{2.5}}$$

```
In [10]: M_abs_sun = 4.75
beta_pic_lum_bol = 10**((M_abs_sun - beta_pic_m_abs_bol)/2.5) * u.Lsun
GDR3_star_lum_bol = 10**((M_abs_sun - GDR3_star_m_abs_bol)/2.5) * u.Lsun

print(beta_pic_lum_bol)
print(GDR3_star_lum_bol)

8.9100616611048 solLum
0.57261050216015 solLum
```

Problem 2d: Calculate the radius of both Beta Pictoris and Gaia DR3 4373465352415301632. How do they compare to the Sun's radius? (6 pts)

2 pts per radius, 2pts for comparison

-0.5 per missing unit

$$L = 4\pi R^2 \sigma_B T_{\rm{eff}}^4$$

$$R^2 = L / (4\pi \sigma_B T_{\rm{eff}}^4)$$

```
In [11]: beta_pic_rad = ((beta_pic_lum_bol/(4 * np.pi * c.sigma_sb * beta_pic_teff**4))**0.5)
GDR3_star_rad = ((GDR3_star_lum_bol/(4 * np.pi * c.sigma_sb * GDR3_star_teff**4))**0.5)

print(beta_pic_rad)
print(GDR3_star_rad)

1.578233750371186 solRad
0.8658425677894298 solRad
```

DR3 star is close to solar while Beta Pic is larger

Problem 2e: Use the M--L relation to estimate the mass of both Beta Pictoris and Gaia DR3 4373465352415301632; compare to other methods; percent difference (8 pts)

2 pts per estimated mass; 2 pts for comparison; 2 pts per percent difference

M--L is: $(M/M_{\odot})^{3.8} = (L/L_{\odot})$ so

$$M/M_{\odot} = (L/L_{\odot})^{1/3.8}$$

```
In [12]: beta_pic_mass = (beta_pic_lum_bol/u.Lsun)**(1/3.8) * u.Msun
GDR3_star_mass = (GDR3_star_lum_bol/u.Lsun)**(1/3.8) * u.Msun
```

```
print(beta_pic_mass)
print(GDR3_star_mass)
```

```
1.7781508748237425 solMass
0.8635326554599483 solMass
```

```
In [13]: beta_pic_mass_spec = 1.75 * u.Msun
GDR3_star_mass_spec = 0.93 * u.Msun
```

```
In [14]: beta_pic_perc_diff = np.abs((beta_pic_mass - beta_pic_mass_spec))/(beta_pic_mas
GDR3_star_perc_diff = np.abs((GDR3_star_mass - GDR3_star_mass_spec))/(GDR3_star

print(beta_pic_perc_diff)
print(GDR3_star_perc_diff)
```

```
0.3989465845214003
1.8529728002919001
```

Problem 3a: Try printing the results table `r`, which shows each star as a separate row. How does the requirement for parallax > 0.2 mas influence the distance of the stars in our table? (4pts)

2pts for printing, 2 pts for statement of relation between parallax and distance

```
In [15]: from astroquery.gaia import Gaia
```

```
In [16]: gaiadr3_table = Gaia.load_table('gaiadr3.gaia_source')
```

```
Retrieving table 'gaiadr3.gaia_source'
```

```
In [17]: job = Gaia.launch_job("select top 5000 "
                                "source_id, phot_g_mean_mag, "
                                "phot_bp_mean_mag, phot_rp_mean_mag, parallax "
                                "from gaiadr3.gaia_source where parallax_over_error > 20
r = job.get_results()
```

```
In [18]: print(r)
```

source_id allax	phot_g_mean_mag mag	phot_bp_mean_mag mag	phot_rp_mean_mag mag	par m

3444412829062210560	15.203718	15.861041	14.400867	0.639025
4343352071				
6369569510812755328	18.305372	20.081676	16.988579	3.10012
2673346884				
5902559231207979008	17.233137	18.138681	16.245045	1.663057
1269187417				
5822406207933947264	15.0276575	15.310797	14.57748	0.457857
0640490629				
2060421091813445504	15.539901	16.113426	14.751871	0.887549
3303315524				
1130547237552438272	16.076492	16.56474	15.475897	0.657031
2533695974				
5535370966536274944	17.023443	17.807999	16.15268	1.003460
1702807633				
4075924930328092928	16.76521	17.73934	15.720893	1.807566
8270434355				
394674474794843520	16.596682	17.230816	15.811058	1.188870
5986579504				
4470357134563239680	15.536937	15.909667	14.995893	0.743395
4403373675				
...	...	...	...	
...				
5298327667020505472	15.864756	16.229574	15.279595	0.560756
0984550689				
3554259886127570048	15.08199	15.420547	14.58145	0.733854
6916012361				
1981023742261690496	15.611056	16.140142	14.911842	0.613899
7056272557				
2180027375422078208	17.767748	18.823011	16.766033	1.514969
5426951488				
3049744783839836800	13.424101	13.922597	12.711549	0.314763
3868540612				
5902087643806938624	14.423031	14.806058	13.863464	0.605028
4455022481				
4085567234951470592	15.013269	15.480605	14.374951	0.59367
0214313298				
4471292612793419776	17.22572	18.665703	16.070482	2.70975
3571452602				
5923741013495434368	15.289953	15.705873	14.699824	0.610176
2191252871				
3038149128054126720	15.4154415	15.724897	14.938237	0.687473
2933117222				
6186365516136907904	14.943881	15.303282	14.41617	0.639527
2694432665				

Length = 5000 rows

```
In [19]: print(r['parallax'].to(u.pc, equivalencies=u.parallax()).max())
print(1/(0.0002))
```

4839.62793294812 pc
5000.0

The maximum distance in parsecs is $1/(0.0002 \text{ as}) = 5000 \text{ pc}$.

Problem 3b: plot a scatter plot that shows the color-magnitude diagram which contains the apparent G magnitude on the y axis and the BP - RP magnitude on the x axis for each source. Be sure to label each axis! (4pts)

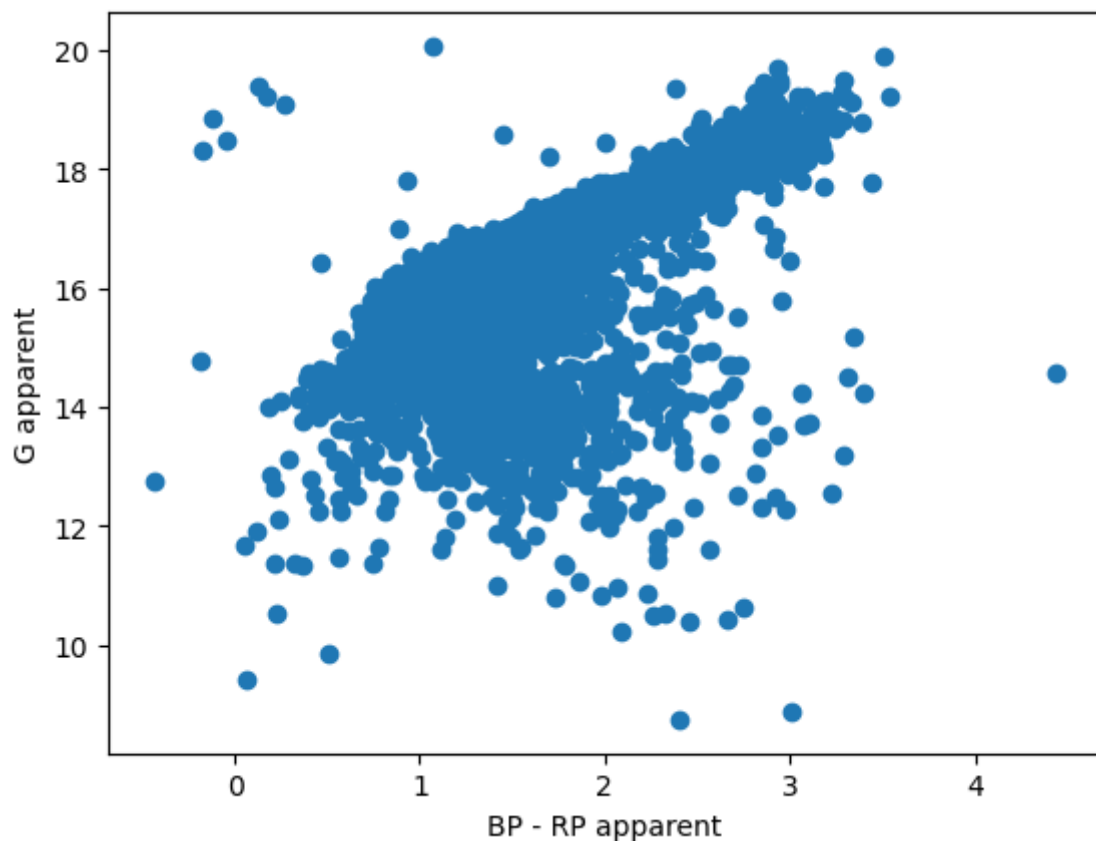
4 pts for the figure

1 pt per missing label

```
In [20]: plt.scatter(r['phot_bp_mean_mag'] - r['phot_rp_mean_mag'], r['phot_g_mean_mag'])

plt.ylabel('G apparent')
plt.xlabel('BP - RP apparent')
```

```
Out[20]: Text(0.5, 0, 'BP - RP apparent')
```



Problem 3c: Convert the parallax to distance and store the distance values in a new column in your results table (2pts)

```
In [21]: r['dist'] = r['parallax'].to(u.pc, equivalencies=u.parallax())
```

Problem 3d: Use distance modulus to get absolute magnitudes and make absolute CMD (10pts)

2 pts per absolute magnitude

4 pts for figure

-1pt per missing axis label

```
In [22]: r['G_abs'] = r['phot_g_mean_mag'].value - 5 * np.log10(r['dist']/(10*u.pc))  
r['BP_abs'] = r['phot_bp_mean_mag'].value - 5 * np.log10(r['dist']/(10*u.pc))  
r['RP_abs'] = r['phot_rp_mean_mag'].value - 5 * np.log10(r['dist']/(10*u.pc))
```

```
In [23]: plt.scatter(r['BP_abs'] - r['RP_abs'], r['G_abs'], s=1)  
  
plt.ylabel('G absolute')  
plt.xlabel('BP - RP absolute')  
plt.ylim(16, -2.5);
```

